

MIAE 215: Computer Programming for Engineers (C++)

Master Course Overview, Study Roadmap & Engineering Strategy

1. Course Information & Engineering Purpose

- **Course Code:** MIAE 215 (formerly MIAE 212 / MECH 215)
- **Title:** *Programming for Mechanical, Industrial, and Aerospace Engineers*
- **Institution:** Gina Cody School of Engineering and Computer Science, Concordia University
- **Core Language:** C++ (ISO Standard C++11/C++14/C++17)

Why C++ for Mechanical, Industrial & Aerospace Engineers?

Students often ask:

"Why are engineers learning C++ instead of Python or MATLAB?"

High-Level Languages (Python, MATLAB)	Systems Language (C++)
-----	-----
• Interpreted (runs through VM) • Garbage collection (unpredictable lag) • High memory & runtime overhead • Slow matrix loops without C-wrappers	• Compiled straight to native machine code • Deterministic manual memory management • Zero-cost abstractions & raw hardware speed • Direct access to CPU registers, RAM, & buses

In industrial engineering, aerospace systems, and advanced robotics, microsecond timing and memory predictability are life-or-death requirements:

1. **Robotics & Autonomous Mechatronics:** Robotic Operating System (ROS / ROS 2) is written predominantly in C++. Flight controllers in drones and aircraft require deterministic control loops with zero garbage-collection pauses.
2. **Computational Fluid Dynamics (CFD) & FEA:** Heavy simulation tools (OpenFOAM, ANSYS solvers, Abaqus user subroutines) crunch billions of matrix calculations; C++ is 50x to 100x faster than pure interpreted Python.
3. **Microcontroller Hardware (Arduino / Embedded Systems):** Microcontrollers have only a few kilobytes of RAM (e.g. ATmega328P has 2 KB SRAM). C++ allows engineers to pack bits into bytes with precision.
4. **Real-Time Digital Manufacturing:** High-speed CNC machines, automated guided vehicles (AGVs), and industrial vision inspection systems rely on native compiled code.

2. The Anatomy of C++: From Code to Silicon

To master C++, an engineer must understand what happens between writing a line of code and the computer executing it:

```
[ Source Code ] (.cpp, .h)
    |
    v
[ Preprocessor ] (Expands #include <iostream>, replaces macros)
    |
    v
[ Compiler ]      (Translates C++ into assembly / machine code)
    |
    v
[ Object File ]   (.obj on Windows / .o on Linux)
    |
    v <----- [ Precompiled Libraries ] (libc, libm, iostream)
[ Linker ]         (Resolves external symbols, functions, memory addresses)
    |
    v
[ Executable ]    (program.exe)
    |
    v
[ OS Loader ]     (Loads binary into Virtual Memory: Stack, Heap, Text)
    |
    v
[ CPU Execution ] (Registers, ALU, Cache, RAM)
```

Key Stages Explained:

1. **Preprocessor:** Text substitution engine. Any line starting with `#` is executed before compilation. `#include <iostream>` literally copies and pastes the entire contents of the `iostream` header into your file.
 2. **Compiler:** Syntactic and semantic parser. Converts human-readable C++ instructions into binary machine instructions, flagging syntax errors, undeclared variables, and type mismatches.
 3. **Linker:** Connects your code with external binary libraries. If you call `sin(x)` or `cout`, the linker finds the precompiled implementation and binds the memory address into your final `.exe`.
 4. **Virtual Memory Layout:**
 - **Text Segment:** Read-only compiled CPU instructions.
 - **Data / BSS:** Global and static variables.
 - **Stack:** Ultra-fast local variables created and destroyed inside functions (LIFO: Last In, First Out).
 - **Heap:** Dynamically allocated memory (`new` / `delete`).
-

3. Semester Topical Roadmap

Phase 1: Foundations, Architecture & Data Types (Weeks 1–3)

- **Week 1: Foundations of Digital Computers & First Program**
 - Structure of a C++ program: `main()`, namespaces, directives, semicolons.
 - Standard I/O streams: `cin >>`, `cout <<`, formatting escape sequences (`\n`, `\t`).
- **Weeks 1–2: Variable Types I & II (Memory Architecture)**
 - Primitive data types: `int`, `float`, `double`, `char`, `bool`.
 - Memory footprints: `sizeof()`, binary limits, overflow/underflow wrap-around.
 - IEEE 754 floating-point numbers: precision, machine epsilon, round-off error, `inf`, `nan`.
 - Type casting: implicit promotion vs. explicit C-style `(int)x` and static casting.
 - Type modifiers: `short`, `long`, `unsigned`, `const`.
 - Hardware architectures: 32/64-bit PC (`sizeof(int)=4`) vs. 8-bit Arduino (`sizeof(int)=2`).

Phase 2: Operators, Control Flow & Numerical Algorithms (Weeks 3–5)

- **Expressions & Operators:**
 - Arithmetic operators (`+`, `-`, `*`, `/`, `%`).
 - **The Integer Division Trap:** Why `1 / 3` evaluates to `0` instead of `0.3333` !
 - Relational and logical operators (`==`, `!=`, `<`, `>`, `&&`, `||`, `!`). Short-circuit evaluation.
- **Selection & Branching:**
 - `if`, `else if`, `else`. Floating-point comparisons using tolerance ϵ (`abs(a - b) < 1e-7`).
 - `switch / case` branching.
- **Repetition & Iterative Algorithms:**
 - `for` loops, `while` loops, `do-while` loops. Loop counters, step sizes, and termination conditions.
 - Numerical algorithms: Successive halving, interval root searching, grid-search function optimization.

Phase 3: Modular Programming & Arrays (Weeks 6–8)

- **Functions & Modular Design:**
 - Function prototypes, definitions, parameters, and return types.
 - Pass-by-value vs. Pass-by-reference (`&`).
 - Scope: local vs. global variables, lifetime, call stack dynamics.
- **Arrays & Matrices:**
 - 1D arrays for vector storage; 2D arrays for engineering matrices.
 - Memory layout of contiguous arrays in row-major order.
 - Array indexing, out-of-bounds safety, and iteration loops.

Phase 4: Pointers, Dynamic Memory & Engineering Applications (Weeks 9–13)

- **Pointers & Memory Addresses:**
 - Address-of operator (`&`) and dereference operator (`*`).
 - Pointer arithmetic, dynamic memory allocation on the heap (`new`, `delete`), avoiding memory leaks.
- **Structures (`struct`) & Engineering Data Records:**
 - Grouping heterogeneous data (e.g., coordinates, sensor readings, aircraft telemetry).

- **File I/O (<fstream>):**
 - Reading experimental sensor data from `.txt` and `.csv` files; writing simulation output.

4. First-Principles Debugging & Exam Strategy

1. Watch Out for Integer Division Traps

- **Mistake:** `double area = 1/2 * base * height;`
- **Why it fails:** In C++, dividing two integers (`1 / 2`) performs integer division and truncates the decimal part, evaluating to `0` ! Thus `0 * base * height = 0.0` .
- **Fix:** Force floating-point promotion by using a decimal literal: `1.0 / 2.0 * base * height` .

2. Never Compare Floating-Point Numbers with `==`

- **Mistake:** `if (z == 0.0)` or `if (sin(3.14159) == 0.0)`
- **Why it fails:** Due to IEEE 754 rounding errors, `sin(3.1415926535)` evaluates to `3.23e-16` , not exact zero. Testing `== 0.0` will evaluate to `false` !
- **Fix:** Use absolute difference with an engineering epsilon:

```
if (std::abs(z) < 1.0e-7) { /* practically zero */ }
```

3. Trace Variables Using Hand Execution Tables

When analyzing exam questions that test code output, never guess! Create an execution table:

Line #	Variable x	Variable y	Output Printed
1	5.0	Uninitialized	-
2	5.0	2.5	-
3	10.0	2.5	10 2.5